

QAOA in Solving Unweighted & Weighted Max-Cut

Julia Huynh

Quantum Approximate Optimization Algorithm (QAOA)

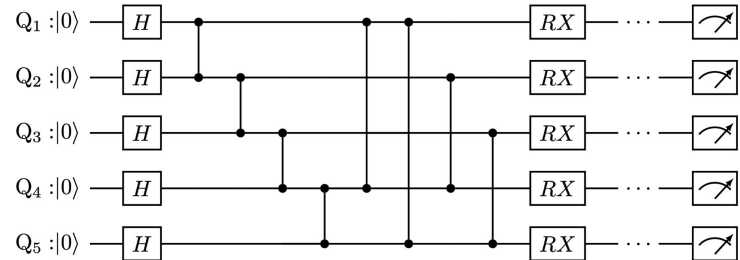
Finds a bitstring that maximizes something by turning the problem into a quantum energy minimization problem

Idea is we make a hamiltonian based off a problem, the ground state is our “optimized” solution, so we try to reach the ground state energy

Useful when: the problem is combinatorial, brute force is difficult, can write a cost function over bitstring answers

Applications:

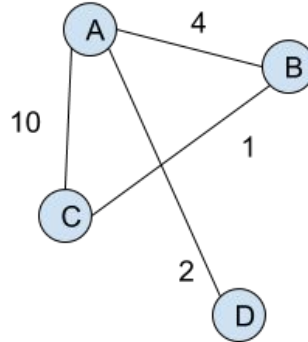
- Graph problems
- Finance/Portfolio Optimization
- Others in Physics, Machine Learning. Etc.



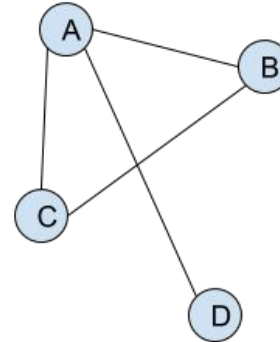
Graph Representation Types

Edge weights can represent, distance, cost, travel, etc.

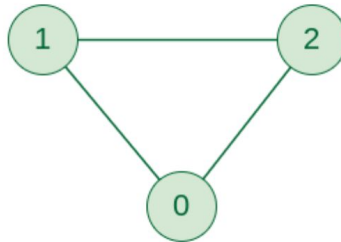
Weighted Graph



Unweighted Graph



Graphs can be represented by adjacency matrices



Undirected Graph



	0	1	2
0		1	1
1	1		1
2	1	1	

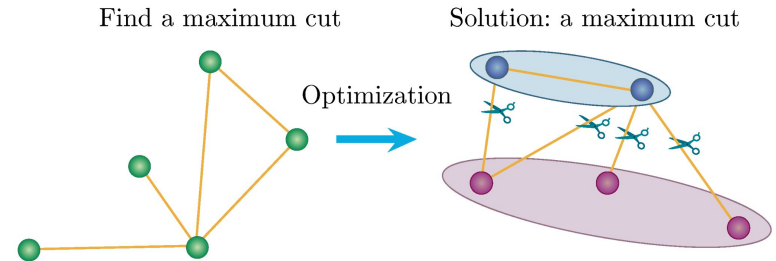
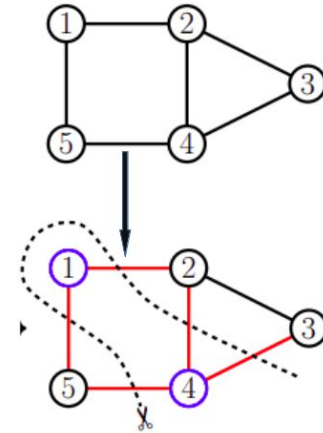
Adjacency Matrix

Max-Cut (NP-Hard Question)

Given a graph of nodes connected with edges, how can you group nodes into two groups as to maximize the amount of edges crossed (cut value)

Examples:

- Friend Networks: How to break up the most friendships in an exam room
- Circuit/Chip Design: How to split circuit components to maximize signal connectivity



QAOA Implementation

For Max-Cut: Bitstrings are indexed by their representative vertices on the graph.

Ex. (0, 1, 0, 1): Vertices 0 and 2 are together & vertices 1 and 3 are together

1. Encode problem as a cost hamiltonian; to optimize, find the “lowest energy state”

Cost Hamiltonian for Unweighted Max-Cut:
$$H = \sum_{(i,j)} \frac{1}{2} (I - Z_i Z_j)$$

2. Initialize a quantum state where every solution is equally likely

$$|+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$$

3. Define a mixer hamiltonian that allows us to move between solutions by flipping each bit independently

$$H_{mixer} = \sum_i X_i$$

4. Alternating unitaries (mixer then cost) p times - this makes a layer

$$|\psi(\gamma, \beta)\rangle = e^{-i\beta H_M} e^{-i\gamma H_C} |+\rangle^{\otimes n}$$

H_cost amplifies “good cuts” & suppresses “bad cuts” through interference

H_mixer spreads amplitudes between bitstrings

5. Optimize parameters gamma & beta to minimize energy, maximizing cut value

- Made my own optimization function
- Also tested out SciPy “Powell” minimize method

6. Apply the QAOA layers with optimized parameters to get the post-QAOA state then measure it by making a distribution and random sampling

Brute Force Method

Iterates through 2^n possible bitstrings given the amount of vertices and finds the best cut-value

Pros:

- Exact solutions

Cons:

- Difficult as number of vertices scale up, increasing dimension size and runtime

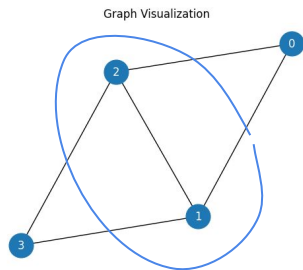
```
# brute force solve that checks all solutions and picks the best one
def brute_force_maxcut(adjacency_matrix):
    n = len(adjacency_matrix) # number of vertices

    # initialize best solutions
    best_x = None
    best_val = -1

    # try all possible bitstrings of length n and finds their cut value
    for x in itertools.product([0, 1], repeat=n):
        value = cut_value(x, adjacency_matrix)
        # if the cut value is better than the current, updates the best solution
        if value > best_val:
            best_val = value
            best_x = x

    # returns the best bitstring and its cut value
    return best_x, best_val
```

QAOA vs. Brute Force for Unweighted Max-Cut

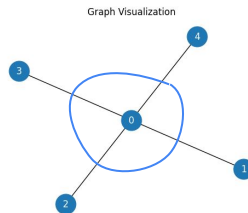


Best QAOA energy: 0.9157235092429842

Top sampled cuts:

(0, 1, 1, 0) -> cut=4, freq=6
 (1, 0, 0, 1) -> cut=4, freq=6
 (1, 1, 0, 0) -> cut=3, freq=31
 (0, 0, 1, 1) -> cut=3, freq=32
 (1, 0, 1, 0) -> cut=3, freq=29

Brute force best: [(0, 1, 1, 0), (1, 0, 0, 1)] cut = 4



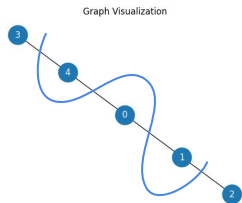
Best performing

Best QAOA energy: 1.0076009182573544

Top sampled cuts:

(1, 0, 0, 0, 0) -> cut=4, freq=43
 (0, 1, 1, 1, 1) -> cut=4, freq=49
 (0, 1, 0, 1, 1) -> cut=3, freq=8
 (1, 0, 0, 1, 0) -> cut=3, freq=4
 (1, 0, 1, 0, 0) -> cut=3, freq=10

Brute force best: [(0, 1, 1, 1, 1), (1, 0, 0, 0, 0)] cut = 4

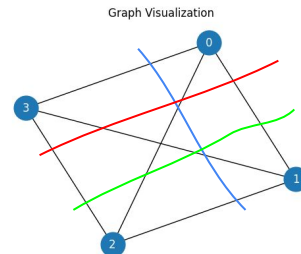


Best QAOA energy: 0.8832658966851661

Top sampled cuts:

(0, 1, 0, 0, 1) -> cut=4, freq=3
 (1, 0, 1, 1, 0) -> cut=4, freq=4
 (0, 1, 1, 0, 1) -> cut=3, freq=3
 (1, 0, 1, 0, 0) -> cut=3, freq=4
 (0, 1, 0, 1, 1) -> cut=3, freq=5

Brute force best: [(0, 1, 0, 0, 1), (1, 0, 1, 1, 0)] cut = 4



Best QAOA energy: 0.9675911988836512

Top sampled cuts:

(0, 1, 0, 1) -> cut=4, freq=32
 (1, 0, 1, 0) -> cut=4, freq=33
 (0, 1, 1, 0) -> cut=4, freq=28
 (1, 1, 0, 0) -> cut=4, freq=28
 (0, 0, 1, 1) -> cut=4, freq=33

Brute force best: [(0, 0, 1, 1), (0, 1, 0, 1), (0, 1, 1, 0), (1, 0, 0, 1), (1, 0, 1, 0), (1, 1, 0, 0)] cut = 4

QAOA Implementation (Weighted)

Goal: Maximize total cut value not number

Similar steps except we update the cost hamiltonian and cut value methods to account for weighted edges

$$H_C = \sum_{(i,j) \in E} w_{ij} \cdot \frac{1}{2}(I - Z_i Z_j)$$

```
# classical max-cut value for weighted
def weighted_cut_value(x, adj):
    value = 0
    n = len(x)

    # iterates over all edges and counts how many edges are cut
    for i in range(n):
        for j in range(i+1, n):
            weight = adj[i][j]
            if weight != 0 and x[i] != x[j]:
                # adds the weight of the edge to the cut value
                value += weight
    return value
```

```
# create the cost hamiltonian for the weighted max-cut problem
def create_weighted_hamiltonian(adj):
    n = len(adj)
    dim = 2**n
    hamiltonian = np.zeros((dim, dim))

    # iterates over each edge and adds the term to the hamiltonian
    for i in range(n):
        for j in range(i+1, n):
            # same as before except now we have weights on the edges
            weight = adj[i][j]
            if weight != 0:
                ops = [np.eye(2)] * n
                ops[i] = Z
                ops[j] = Z

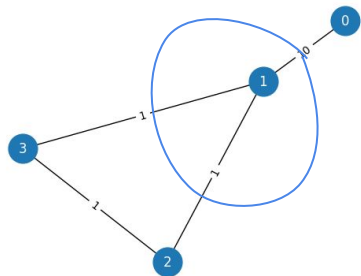
                Zij = ops[0]
                for op in ops[1:]:
                    Zij = np.kron(Zij, op)

                hamiltonian += weight * 0.5 * (np.eye(dim) - Zij)

    return hamiltonian
```

QAOA vs. Brute Force for Weighted Max-Cut

Graph Visualization



Best QAOA energy: 1.5145041175773253

Top sampled cuts:

(1, 0, 1, 1) -> cut value = 12, freq = 1
(0, 1, 0, 0) -> cut value = 12, freq = 3
(0, 1, 0, 1) -> cut value = 12, freq = 1
(1, 0, 0, 0) -> cut value = 10, freq = 3
(0, 1, 1, 1) -> cut value = 10, freq = 6

Brute force best: [(0, 1, 0, 0), (0, 1, 0, 1), (0, 1, 1, 0), (1, 0, 0, 1), (1, 0, 1, 0), (1, 0, 1, 1)] cut value = 12

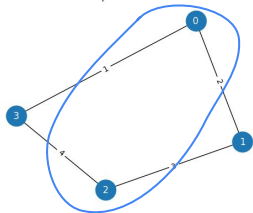
Best QAOA energy: 2.329288720884912

Top sampled cuts:

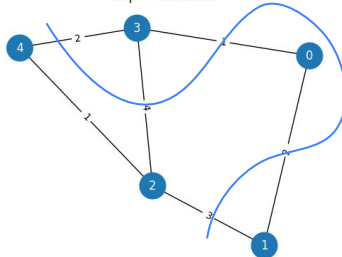
(1, 0, 1, 0) -> cut value = 10, freq = 5
(0, 1, 0, 1) -> cut value = 10, freq = 6
(0, 0, 1, 0) -> cut value = 7, freq = 1
(0, 1, 1, 0) -> cut value = 6, freq = 36
(1, 0, 0, 1) -> cut value = 6, freq = 36

Brute force best: [(0, 1, 0, 1), (1, 0, 1, 0)] cut value = 10

Graph Visualization



Graph Visualization



Best QAOA energy: 3.3581792127263883

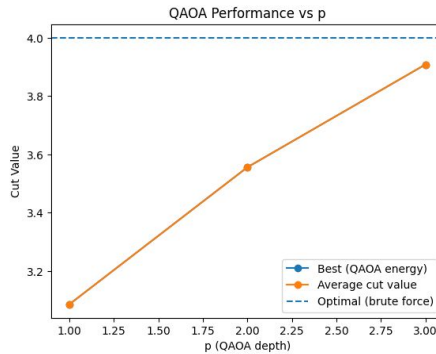
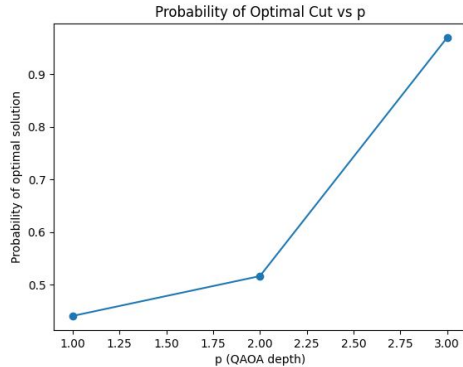
Top sampled cuts:

(1, 0, 1, 0, 1) -> cut value = 12, freq = 2
(0, 1, 0, 1, 0) -> cut value = 12, freq = 1
(0, 1, 0, 1, 1) -> cut value = 11, freq = 6
(1, 0, 1, 0, 0) -> cut value = 11, freq = 4
(1, 1, 0, 1, 0) -> cut value = 9, freq = 6

Brute force best: [(0, 1, 0, 1, 0), (1, 0, 1, 0, 1)] cut value = 12

Parameter Analysis of p (QAOA Layer Depth)

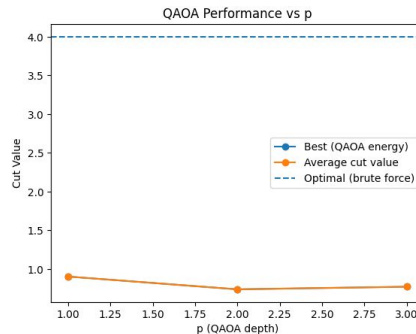
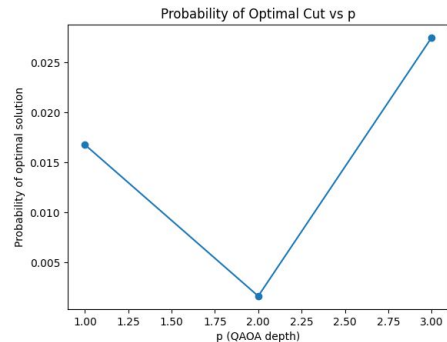
Parameter Optimization Using SciPy:



SciPy's optimization is more exact than mine, but overall mine still worked for Max-Cut

Although a larger p means better results, it makes it harder to optimize because there's more parameters

My QAOA Optimization Method:

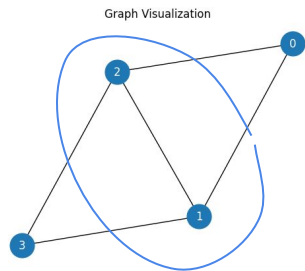


Parameter # = $2 * p$

This means as graphs get larger, these parameters have to control larger quantum states which is difficult to optimize

Extra: after fixing the bug (aka didn't put a negative on a line...)

QAOA vs. Brute Force for Unweighted Max-Cut

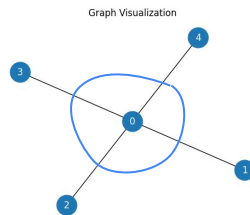


Best QAOA energy: 3.2341642918829456

Top sampled cuts:

(1, 0, 0, 1) -> cut=4, freq=161
(0, 1, 1, 0) -> cut=4, freq=159
(1, 1, 0, 1) -> cut=3, freq=60
(0, 0, 1, 0) -> cut=3, freq=70
(1, 0, 1, 1) -> cut=3, freq=63

Brute force best: [(0, 1, 1, 0), (1, 0, 0, 1)]
cut = 4

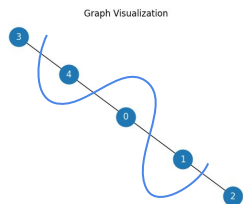


Best QAOA energy: 2.992566146773214

Top sampled cuts:

(0, 1, 1, 1, 1) -> cut=4, freq=220
(1, 0, 0, 0, 0) -> cut=4, freq=229
(0, 1, 1, 1, 0) -> cut=3, freq=48
(0, 0, 1, 1, 1) -> cut=3, freq=47
(1, 0, 0, 0, 1) -> cut=3, freq=34

Brute force best: [(0, 1, 1, 1, 1), (1, 0, 0, 0, 0)]
cut = 4



Best QAOA energy: 3.1134743000222205

Top sampled cuts:

(0, 1, 0, 0, 1) -> cut=4, freq=153
(1, 0, 1, 1, 0) -> cut=4, freq=159
(1, 0, 1, 0, 1) -> cut=3, freq=74
(0, 1, 1, 0, 1) -> cut=3, freq=49
(1, 1, 0, 1, 0) -> cut=3, freq=85

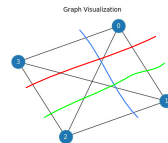
Brute force best: [(0, 1, 0, 0, 1), (1, 0, 1, 1, 0)]
cut = 4

Best QAOA energy: 3.6970569948841066

Top sampled cuts:

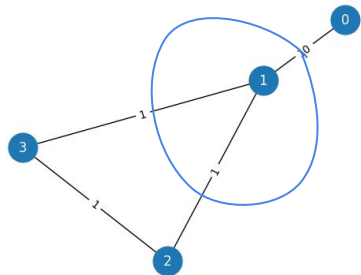
(1, 0, 0, 1) -> cut=4, freq=112
(1, 0, 1, 0) -> cut=4, freq=152
(0, 1, 1, 0) -> cut=4, freq=120
(0, 0, 1, 1) -> cut=4, freq=124
(0, 1, 0, 1) -> cut=4, freq=123

Brute force best: [(0, 0, 1, 1), (0, 1, 0, 1), (0, 1, 1, 0), (1, 0, 0, 1), (1, 0, 1, 0), (1, 1, 0, 0)]
cut = 4



QAOA vs. Brute Force for Weighted Max-Cut

Graph Visualization



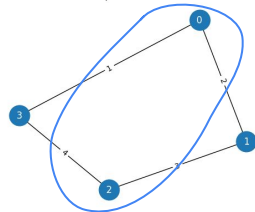
Best QAOA energy: 11.520415565119656

Top sampled cuts:

(0, 1, 0, 0) → cut value = 12, freq = 123
 (1, 0, 1, 1) → cut value = 12, freq = 123
 (1, 0, 0, 1) → cut value = 12, freq = 153
 (1, 0, 1, 0) → cut value = 12, freq = 170
 (0, 1, 1, 0) → cut value = 12, freq = 137

Brute force best: [(0, 1, 0, 0), (0, 1, 0, 1), (0, 1, 1, 0), (1, 0, 0, 1), (1, 0, 1, 0), (1, 0, 1, 1)] cut value = 12

Graph Visualization



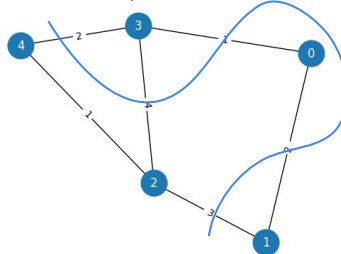
Best QAOA energy: 7.652447632631012

Top sampled cuts:

(1, 0, 1, 0) → cut value = 10, freq = 228
 (0, 1, 0, 1) → cut value = 10, freq = 223
 (0, 0, 1, 0) → cut value = 7, freq = 64
 (1, 1, 0, 1) → cut value = 7, freq = 79
 (0, 1, 1, 0) → cut value = 6, freq = 89

Brute force best: [(0, 1, 0, 1), (1, 0, 1, 0)] cut value = 10

Graph Visualization



Best QAOA energy: 9.209052666246023

Top sampled cuts:

(0, 1, 0, 1, 0) → cut value = 12, freq = 132
 (1, 0, 1, 0, 1) → cut value = 12, freq = 114
 (1, 0, 1, 0, 0) → cut value = 11, freq = 85
 (0, 1, 0, 1, 1) → cut value = 11, freq = 107
 (1, 1, 0, 1, 0) → cut value = 9, freq = 45

Brute force best: [(0, 1, 0, 1, 0), (1, 0, 1, 0, 1)] cut value = 12

Using my optimization for p analysis

